

Merge conflicts

When two branches edit the same lines, Git cannot pick a winner automatically

Read the markers, then resolve

- Look for less-than signs marking HEAD, your current branch
- Edit the file until the result is what you want and every marker is gone
- Stage the fixed file, then commit to complete the merge
- In RTL work, conflicts often mean two people touched the same module, coordinate when you

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Conflicts

INTERACTIVE TOOL

Merge conflict resolver

Practice reading conflict markers, choosing ours/theirs/manual, and finishing with a clean file — the skill behind `git merge` pain.

Starter example: counter WIDTH conflict — keep enable and WIDTH=16.

Load starter example

Challenges 0 / 22 cleared

Counter WIDTH + enable: Merge: keep WIDTH=16 and logic enable; remove all markers.

Show hintReset conflictTake ours (HEAD)Take theirsShow suggested mergeCheck resolutionNext

Not checked

Counter WIDTH + enableTake ours onlyTake theirs onlyComment conflictReset polarityKeep both params

Include guard textassign vs alwaysPackage importTimescalewire vs regClock nameTwo hunks

Added on featureDeleted on featureLicense headerTB clock periodcase defaultgenerate ifMarkers only

always_ffANSI ports

ours (HEAD)

module counter;
// main branch
parameter WIDTH = 8;
logic [WIDTH-1:0] q;
endmodule

theirs (feature)

module counter;
// feature branch
parameter WIDTH = 16;
logic [WIDTH-1:0] q;
logic enable;
endmodule

working tree

Merge conflict resolver starter

Real Git practice

```
wsl - module10-git-conflicts/examples/conflicts

# Track A - real Unix
# real shell session (Track A)

# practice repo: /tmp/git-conf-MWYSuJ

$ git init

$ echo "shared" > notes.md

$ git add notes.md

$ git commit -m "Initial"

$ git branch -M main

$ git checkout -b feature/conflict-test

$ echo "Line from feature" > notes.md

$ git add notes.md

$ git commit -m "Change on feature"

$ git checkout main

$ echo "Line from main" > notes.md

$ git add notes.md

$ git commit -m "Change on main"

$ git merge feature/conflict-test
Auto-merging notes.md
CONFLICT (content): Merge conflict in notes.md
Automatic merge failed; fix conflicts and then commit the result.

$ cat notes.md
<<<<<< HEAD
Line from main
=====
Line from feature
>>>>>> feature/conflict-test

$ echo "Resolved line" > notes.md

$ git add notes.md

$ git commit -m "Resolve merge conflict"

$ git log --oneline --graph --all
* 864f342 Resolve merge conflict
| \
| * 5bb45a9 Change on feature
* | 33cd954 Change on main
| /
* 712590c Initial
```

Real shell — create, resolve, and commit a merge conflict

Real Git practice — try these

```
# git init — create a new repository here  
git init
```

```
# echo ... > notes.md — shared starting content  
echo "shared" > notes.md
```

```
git add notes.md
```

```
git commit -m "Initial"
```

```
git branch -M main
```

```
# git checkout -b feature/conflict-test — branch for a conflicting  
edit
```

```
git checkout -b feature/conflict-test
```

Pitfalls to watch

- Do not commit while conflict markers remain in the file
- Do not panic-delete both sides, read what each branch changed
- Pull and merge main into your feature branch early to shrink conflicts before review
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load the starter and clear a few conflict regions
- On real Git, create a small conflict, resolve it, and commit the merge
- When you are ready